

Reconocimiento de Dígitos Manuscritos en Línea

Vectores de Cuantificación, Modelos Ocultos de Markov
y Sistemas en Ensemble

e-BioDigit DB – 93 usuarios, 10 dígitos

Miguel Ángel Calderón

Marcos González

María Zürcher

Álvaro Sáiz

Gonzalo Cabrera

Yago Clérigo

Blas Jesús Acosta

5 de mayo de 2026

Índice

1. Vector Quantization (VQ)	2
1.1. Descripción del método	2
1.2. Hiperparámetros y variantes exploradas	2
1.3. Resultados	2
1.4. Conclusiones del VQ	5
2. Modelos Ocultos de Markov (HMM)	6
2.1. Descripción del método	6
2.2. Hiperparámetros y variantes exploradas	6
2.3. Resultados	6
2.4. El coste computacional del GMMHMM y el LOO	9
2.5. Conclusiones del HMM	9
3. Sistemas en Ensemble	11
3.1. Motivación: el HMM y el VQ se equivocan de forma distinta	11
3.2. Ensemble paralelo	11
3.3. Ensemble serial	12
3.4. Conclusiones de los ensembles	15

1. Vector Quantization (VQ)

1.1. Descripción del método

Código: la implementación base está en `Entrega2/VQ/implementacion_VQ.py`, que define los algoritmos KMeans, MiniBatchKMeans y LBG sobre el extractor de características local.

La idea de VQ es sencilla: para cada dígito se construye un *codebook* mediante KMeans, es decir, un conjunto de centroides que resume el espacio de vectores de características de ese dígito. A la hora de clasificar una muestra nueva se calcula, para cada dígito, la distorsión media entre los vectores de la muestra y los centroides más cercanos de ese codebook. El dígito ganador es el que produce la distorsión más baja.

El extractor de características local proporciona hasta 23 dimensiones por punto de la trayectoria. Para el sistema base se utilizaron 7 de ellas: posición (x, y) , desplazamiento $(\Delta x, \Delta y)$, velocidad v , $\sin(\alpha)$ y $\cos(\alpha)$ (siendo α el ángulo instantáneo). La búsqueda posterior identificó que el subconjunto `pos_ang_curv` de 5 dimensiones $(x, y, \sin \alpha, \cos \alpha, \Delta \theta)$ es más discriminativo.

1.2. Hiperparámetros y variantes exploradas

Código: la búsqueda en rejilla está en `Entrega2/VQ/busqueda_VQ.py`, que recorre las 144 configuraciones y deja los resultados en `Entrega2/VQ/resultados_VQ/`.

Se realizó una búsqueda sobre 144 configuraciones combinando:

- **Algoritmo de cuantificación:** KMeans, MiniBatchKMeans y LBG.
- **Número de centroides por dígito:** 8, 16, 32, 64, 128 y 256.
- **Subconjunto de características:** `pos_ang_curv` (5D), `med` (12D), `kin_deriv` y otros subconjuntos del extractor.

La configuración ganadora fue MiniBatchKMeans con 128 centroides y el subconjunto `pos_ang_curv` (5D), con una precisión de validación del 96.9%. El rendimiento escala bien con el número de centroides hasta 128, tras lo cual la ganancia marginal cae.

1.3. Resultados

Código: las curvas DET y los EER se calculan en `Entrega3/generar_det_nuevos.py`, que carga los resultados del VQ y produce los plots de `Entrega3/metricas/det/`.

La Tabla 1 recoge la precisión y el Equal Error Rate (EER) de la curva DET para las dos versiones del VQ y los tres escenarios de evaluación: N=74 (74 usuarios de entrenamiento, 19 de test), N=47 y Leave-One-Out (LOO).

Cuadro 1: Precisión (%) y EER (%) del sistema VQ.

Sistema	N=74		N=47		LOO	
	Acc.	EER	Acc.	EER	Acc.	EER
VQ base (K=32, 7D)	95.6	7.70	96.1	7.01	95.8	7.23
VQ opt. (K=128, 5D, MBKMeans)	97.0	1.84	97.1	1.93	96.9	1.90

La Figura 1 muestra las curvas DET del VQ base frente al mejor HMM de la Entrega 1 para los tres escenarios. El VQ base ya supera al HMM en EER para N=74 y N=47; en LOO el HMM se degrada mucho más, lo que revela que el VQ generaliza mejor a usuarios no vistos.

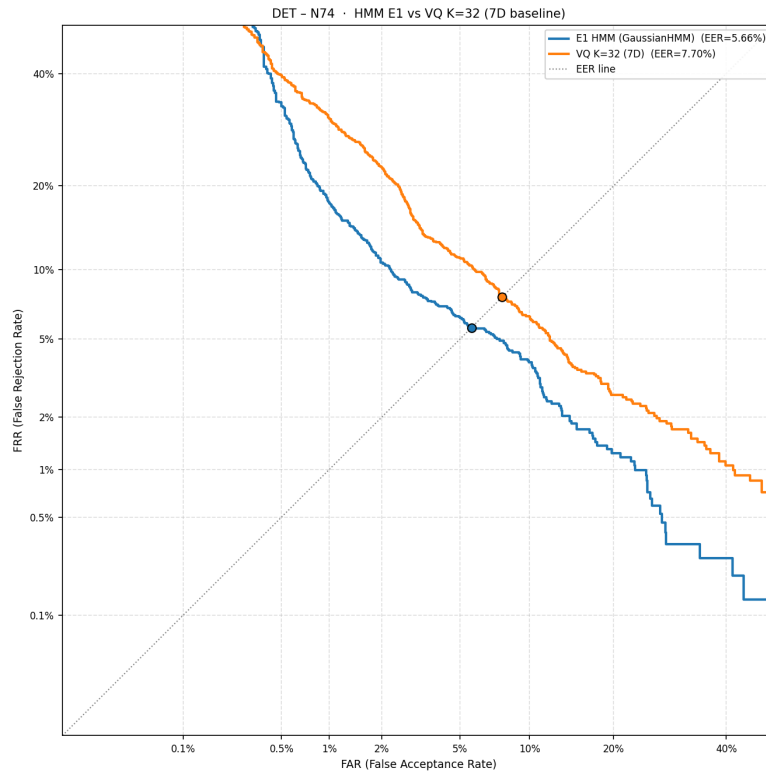


Figura 1: Curvas DET (N=74): HMM (E1) frente a VQ base (K=32, 7D).

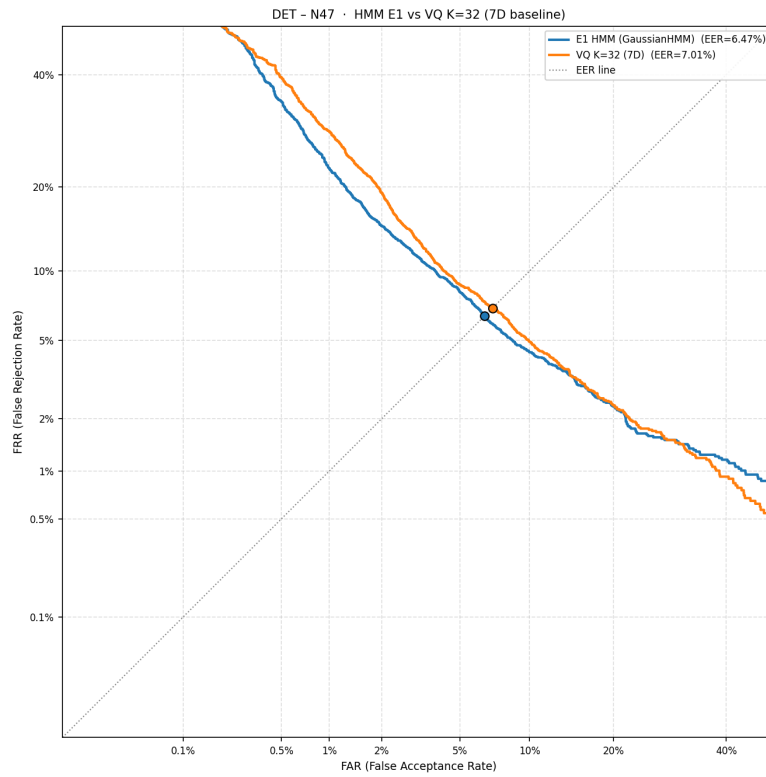


Figura 2: Curvas DET (N=47): HMM (E1) frente a VQ base (K=32, 7D).

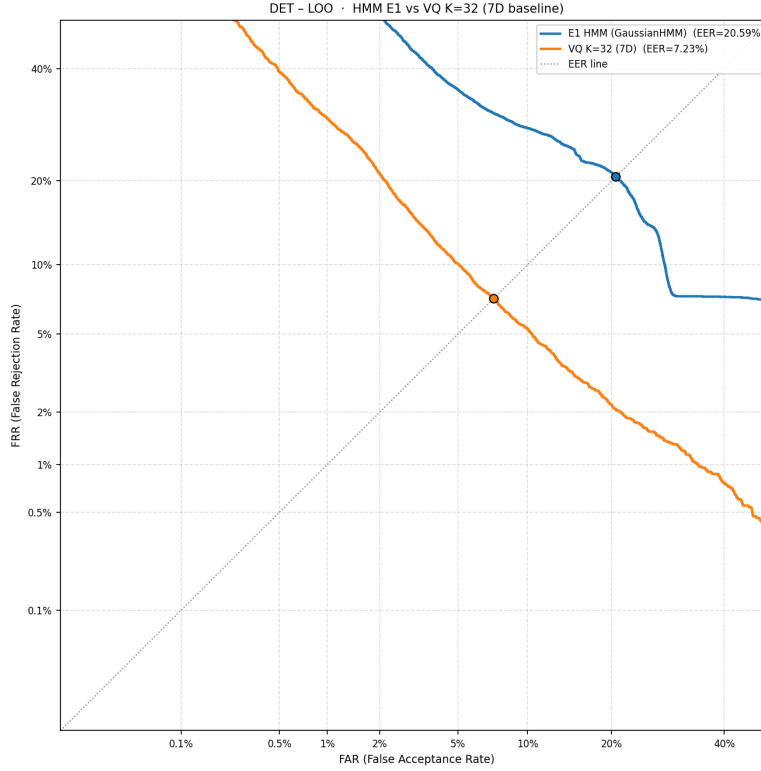


Figura 3: Curvas DET (LOO): HMM (E1) frente a VQ base (K=32, 7D).

1.4. Conclusiones del VQ

El VQ es el sistema más eficiente en relación a su coste: el entrenamiento de un codebook por dígito con MiniBatchKMeans tarda segundos, y sin embargo obtiene el mejor EER individual del estudio (1.84–1.93 % según el escenario). La clave es la elección de las características: el subconjunto de 5 dimensiones orientado a posición y curvatura captura lo esencial del trazo manuscrito sin ruido extra. El principal límite del VQ es que no modela la dinámica temporal de la escritura; cada punto se trata de forma independiente, lo que impide distinguir dígitos que comparten formas locales pero difieren en el orden de los trazos.

2. Modelos Ocultos de Markov (HMM)

2.1. Descripción del método

Código: el HMM gaussiano simple (E1) está en `Entrega1/clasificador_digitos.py` (preprocesado, extracción de características y entrenamiento). El GMMHMM (E2) está en `Entrega2/HMM/clasificador_digitos_v3.py` (función `entrenar_gmmhmm_digito`).

Un HMM de izquierda a derecha modela la secuencia temporal de vectores de características como una cadena de estados ocultos que solo puede avanzar hacia la derecha o quedarse en el mismo estado (autolazos). Cada estado emite vectores con una distribución gaussiana. Para clasificar, se calcula la log-verosimilitud de la secuencia bajo el modelo de cada dígito y se elige el dígito con mayor verosimilitud.

En la Entrega 2 se pasó a un HMM con mezcla de Gaussianas por estado (GMMHMM), lo que permite capturar distribuciones multimodales dentro de cada estado y modelar con mayor precisión los distintos estilos de escritura.

El preprocesado de la trayectoria incluye suavizado Savitzky-Golay, remuestreo equiespaciado en arco a 80 puntos, centrado y escalado a $[-1, 1]$, y normalización Z-score sobre el conjunto de entrenamiento.

2.2. Hiperparámetros y variantes exploradas

Código: la búsqueda de hiperparámetros del HMM E1 se ejecuta desde `Entrega1/ejecutar_parcial.py` y `Entrega1/ejecutar_loo.py`. La del GMMHMM (E2) está integrada en `Entrega2/HMM/clasificador_digitos_v3.py`.

Para el **HMM gaussiano simple (Entrega 1)** se realizó una búsqueda sobre:

- Número de estados: 3, 5, 7, 10 y 15.
- Tipo de covarianza: `full`, `diag`, `tied`, `spherical`.
- Subconjunto de características: `min` (7D), `med` (12D) y `full` (21D).
- Probabilidad de autolazo: 0.3, 0.5, 0.6, 0.7 y 0.9.

La mejor configuración resultó ser 7 estados, covarianza diagonal, subconjunto `med` (12D) y probabilidad de autolazo 0.6, alcanzando una precisión del 92.9 % en N=74.

Para el **GMMHMM (Entrega 2)** se mantuvo la misma topología y se añadió la búsqueda sobre el número de componentes por mezcla (`n_mix` $\in \{1, 2, 3\}$). La mejor configuración fue `n_mix=2` con el resto de hiperparámetros iguales. Con entrenamiento completo (100 iteraciones EM, 10 reinicios por dígito) alcanzó un 93.0 % en N=74 y un 93.3 % en N=47, con EER de 3.09 % y 4.78 % respectivamente.

2.3. Resultados

Cuadro 2: Precisión (%) y EER (%) de los sistemas HMM.

Sistema	N=74		N=47		LOO	
	Acc.	EER	Acc.	EER	Acc.	EER
HMM gaussiano (E1)	92.9	5.66	89.8	6.47	n.a.	20.59
GMMHMM (E2, bien entrenado)	93.0	3.09	93.3	4.78	80.5*	17.38*

* El LOO del GMMHMM se realizó con 5-fold y parámetros reducidos (80 iter., 6 reinicios); ver texto.

La Figura 4 añade el GMMHMM y el VQ optimizado a las curvas de la sección anterior, permitiendo comparar los cuatro sistemas base.

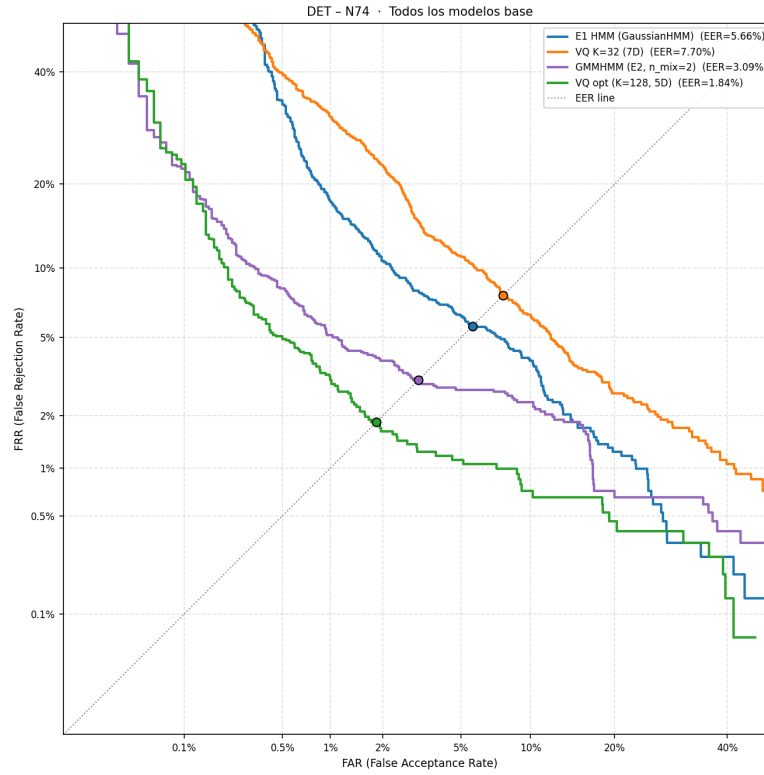


Figura 4: Curvas DET (N=74): todos los modelos individuales.

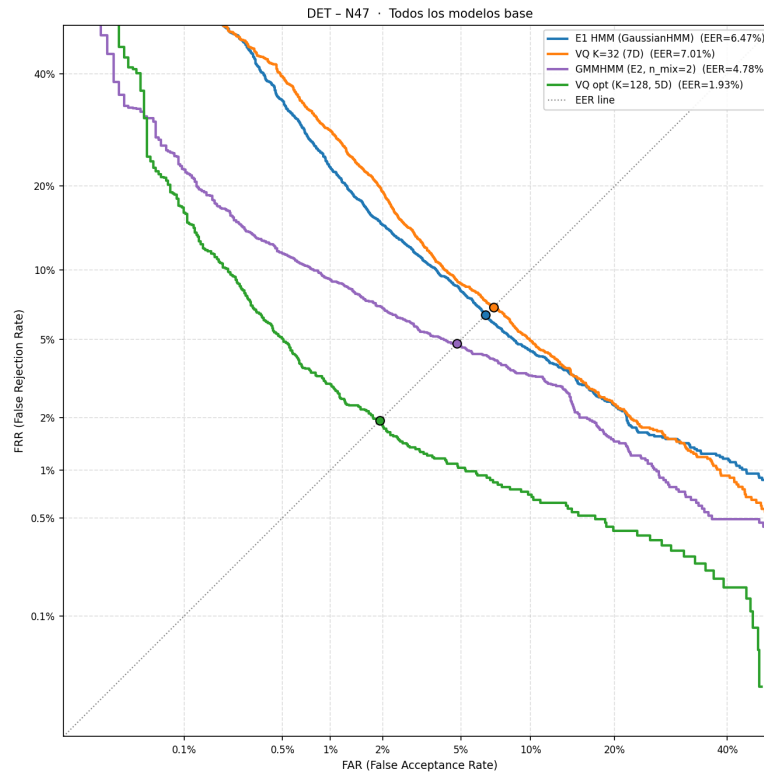


Figura 5: Curvas DET (N=47): todos los modelos individuales.

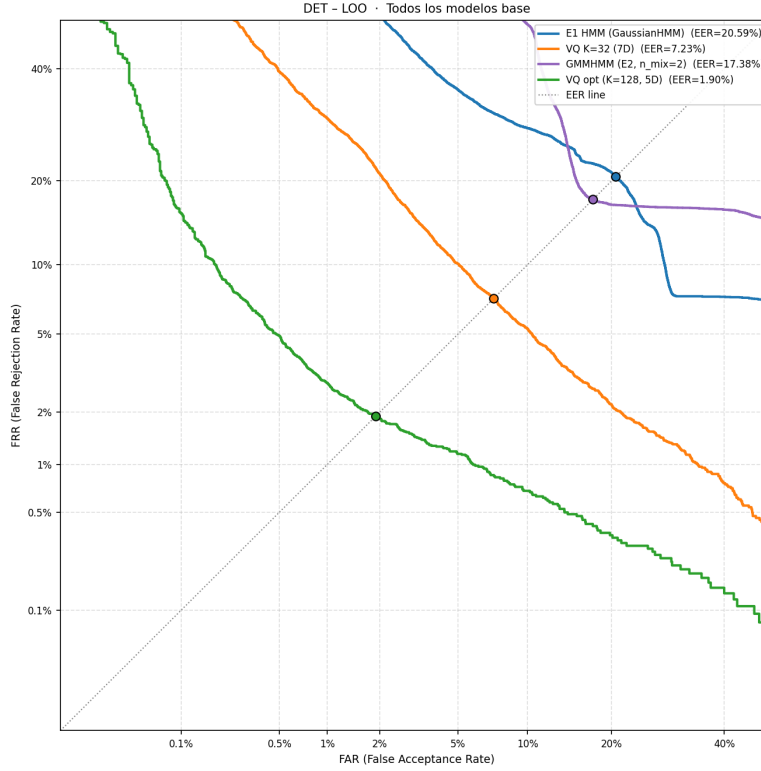


Figura 6: Curvas DET (LOO): todos los modelos individuales.

2.4. El coste computacional del GMMHMM y el LOO

Código: el reentrenamiento completo del GMMHMM (100 iter., 10 reinicios) para los escenarios $N=74$ y $N=47$ se hace en `Entrega3/train_hmm_vq_splits.py`, que guarda los resultados en `Entrega3/Paralel/resultados/ensemble_{N74,N47}_full.json`. La aproximación K5-fold del LOO está en `Entrega3/kfold_hmm_vq.py` (salida en `ensemble_K5fold.json`).

El algoritmo EM del GMMHMM converge de forma significativamente más lenta que el del HMM gaussiano simple. En la máquina de trabajo, entrenar los 10 modelos (uno por dígito) con 100 iteraciones y 10 reinicios requiere aproximadamente **6 horas para $N=74$** . El LOO clásico implicaría repetir esto 93 veces, lo que supone unas 444 horas (18 días), un coste inasumible.

Como alternativa, se utilizó una validación cruzada con **5 pliegues por usuario** (K5-fold), que reduce el número de entrenamientos a 5. Con parámetros algo más ajustados (80 iter., 6 reinicios) el tiempo total bajó a unas 14–15 horas. El resultado del LOO aproximado (80.5 % de precisión) es inferior al de $N=74$ porque cada fold dispone de menos datos de entrenamiento relativos y porque los parámetros de entrenamiento son más limitados.

2.5. Conclusiones del HMM

El GMMHMM mejora claramente al HMM gaussiano simple cuando se entrena con los recursos adecuados. La mejora en EER respecto al HMM de la Entrega 1 es notable en los escenarios con conjunto de entrenamiento fijo ($N=74$ y $N=47$). Sin embargo, el VQ optimizado sigue siendo superior en todos los escenarios, incluso siendo un modelo mucho más simple. El HMM añade valor al sistema por una razón distinta: sus errores no coinciden exactamente con los del VQ, lo que abre la puerta a combinaciones en ensemble que pueden superar a cualquiera de los dos por separado.

3. Sistemas en Ensemble

3.1. Motivación: el HMM y el VQ se equivocan de forma distinta

Código: el scatter por usuario se genera en `Entrega2/comparacion/` (al comparar HMM y VQ por usuario).

Antes de describir los ensembles conviene ver si tiene sentido combinar HMM y VQ. La Figura 7 muestra la precisión por usuario de ambos sistemas en el escenario N=74. Cada punto es un usuario de test; el eje horizontal recoge la precisión del HMM y el vertical la del VQ.

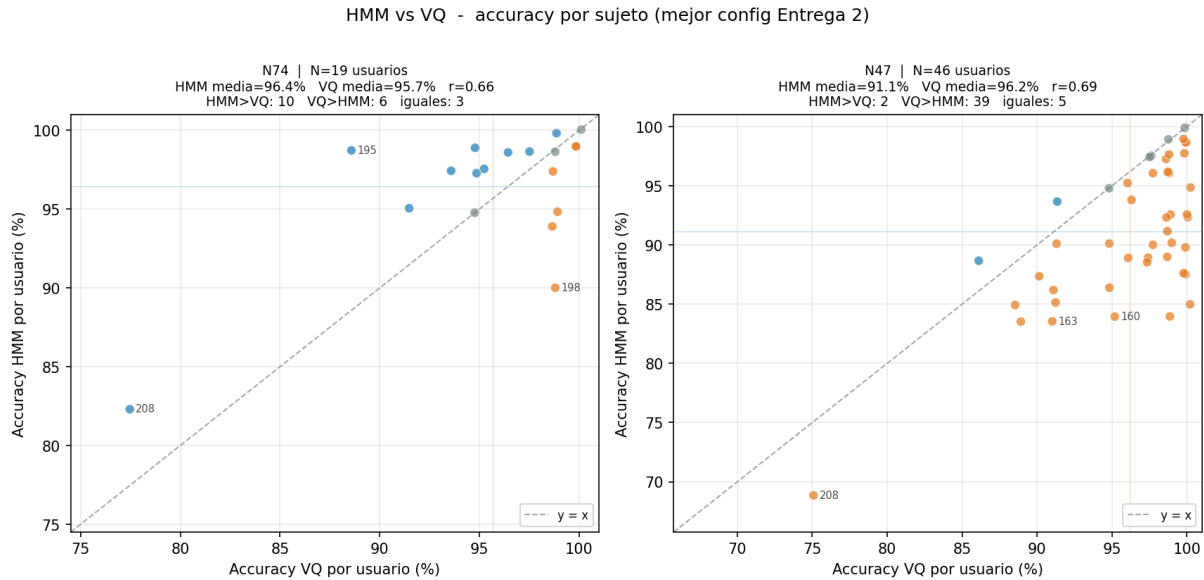


Figura 7: Precisión por usuario: HMM (eje X) frente a VQ (eje Y) en N=74. Los puntos fuera de la diagonal indican usuarios en los que un modelo acierta y el otro falla.

Si ambos modelos cometieran los mismos errores, los puntos estarían sobre la diagonal y no habría nada que ganar combinándolos. El hecho de que existan puntos alejados de la diagonal indica que hay usuarios para los que el HMM falla pero el VQ acierta (y viceversa). Esto sugiere que una combinación inteligente puede recuperar parte de esos errores.

El hecho de que los errores de ambos modelos no coincidan implica que existe margen real para mejorar combinando sus predicciones.

3.2. Ensemble paralelo

Código: el ensemble paralelo con sus reglas de fusión (`agreement`, `soft`, `conf_weighted`, `margin_weighted`) está en `Entrega3/Paralel/ensemble_paralelo.py`. La versión con el GMMHMM bien entrenado se obtiene en `Entrega3/train_hmm_vq_splits.py`.

En el ensemble paralelo, el HMM y el VQ clasifican cada muestra de forma independiente y sus puntuaciones se fusionan según distintas reglas:

agreement: si ambos modelos coinciden, se usa esa predicción; si no, se elige el modelo con mayor confianza en la clase top-1.

soft: promedio simple de las probabilidades softmax de ambos modelos, normalizadas por muestra para hacer las escalas comparables.

conf_weighted: ponderación por la confianza top-1 de cada modelo (el que está más seguro tiene más peso).

margin_weighted: ponderación por el margen entre la clase más probable y la segunda, que refleja mejor cuánto se destaca la decisión de cada modelo.

La regla **margin_weighted** es la que consigue mejores resultados en todos los escenarios; es también la única que no requiere información futura (a diferencia del oráculo, que conoce las etiquetas reales).

Cuadro 3: Precisión (%) y EER (%) del ensemble paralelo (**margin_weighted**).

Sistema	N=74		N=47		LOO (K5)	
	Acc.	EER	Acc.	EER	Acc.	EER
Paralelo margin_weighted	97.6	1.69	97.3	1.74	97.0	2.30

3.3. Ensemble serial

Código: la cascada serial original (HMM $\rightarrow$ VQ en espacio LL) está en `Entrega3/Serial/serial_hmm_vq.py`. La actualización con el GMMHMM bien entrenado se hace con `Entrega3/Serial/update_serial_full_hmm.py`. La cascada serial VQ-dominante está implementada como función `serial_vq_dominant` en `Entrega3/generar_det_nuevos.py`.

En la cascada serial original (Entrega 3), el HMM genera un vector de 10 log-verosimilitudes que se usa como entrada a un clasificador VQ en ese espacio de 10 dimensiones. La idea es que si el HMM funciona bien, la dimensión correspondiente al dígito correcto tendrá el valor más alto, y el VQ aprenderá esos patrones. Aun cuando se utiliza el GMMHMM bien entrenado (100 iteraciones, 10 reinicios), la cadena resulta poco fiable: el VQ en el espacio de log-verosimilitudes degrada respecto al baseline del propio HMM, con EER del 11.5% en N=74 y el 13.1% en N=47. El motivo es que el VQ no añade información nueva sobre las propias verosimilitudes y, al introducir un modelo intermedio entrenado, propaga sus errores en lugar de corregirlos.

Para mejorar este resultado se diseñó una **cascada VQ-dominante**: el VQ actúa como primera etapa y el HMM solo contribuye en las muestras en las que el VQ está inseguro. Concretamente:

1. Se calcula la distribución de probabilidad del VQ sobre las 10 clases.
2. Se mide la incertidumbre del VQ combinando su entropía normalizada y la falta de margen entre la clase más probable y la segunda.
3. El score final es $\alpha \cdot p_{VQ} + (1 - \alpha) \cdot p_{HMM}$, donde $\alpha \in [0,80, 1,00]$ crece con la confianza del VQ.
4. El HMM nunca aporta más de un 20% del peso total.

Con esta estrategia el EER del serial baja hasta el 1.73% en N=74 y el 1.66% en N=47, superando al VQ base (K=32, 7D) y acercando el serial al ensemble paralelo.

Cuadro 4: Comparativa EER (%) de todos los sistemas de ensemble.

Sistema	N=74	N=47	LOO (K5)
Serial original (HMM $\rightarrow$ VQ en espacio LL)	11.46	13.05	12.34
Serial VQ-dominante	1.73	1.66	1.94
Paralelo margin_weighted	1.69	1.74	2.30

La Figura 8 muestra las curvas DET de los tres sistemas de ensemble.

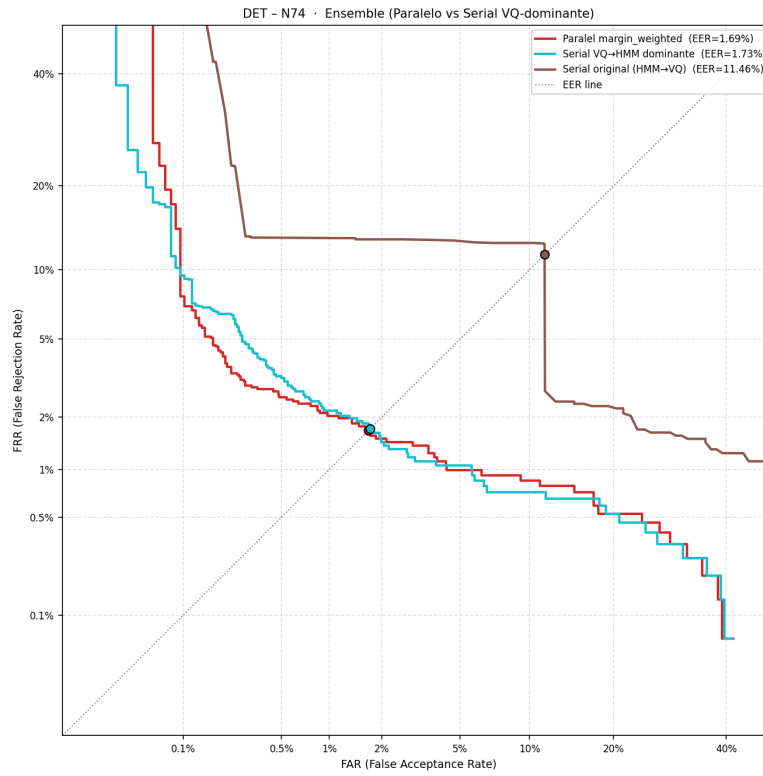


Figura 8: Curvas DET (N=74): sistemas en ensemble.

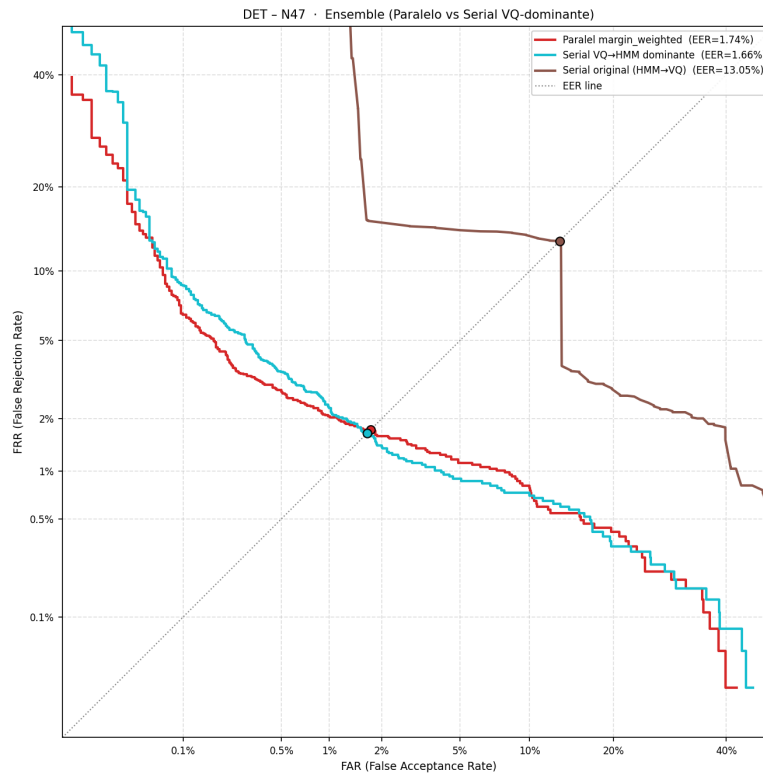


Figura 9: Curvas DET (N=47): sistemas en ensemble.

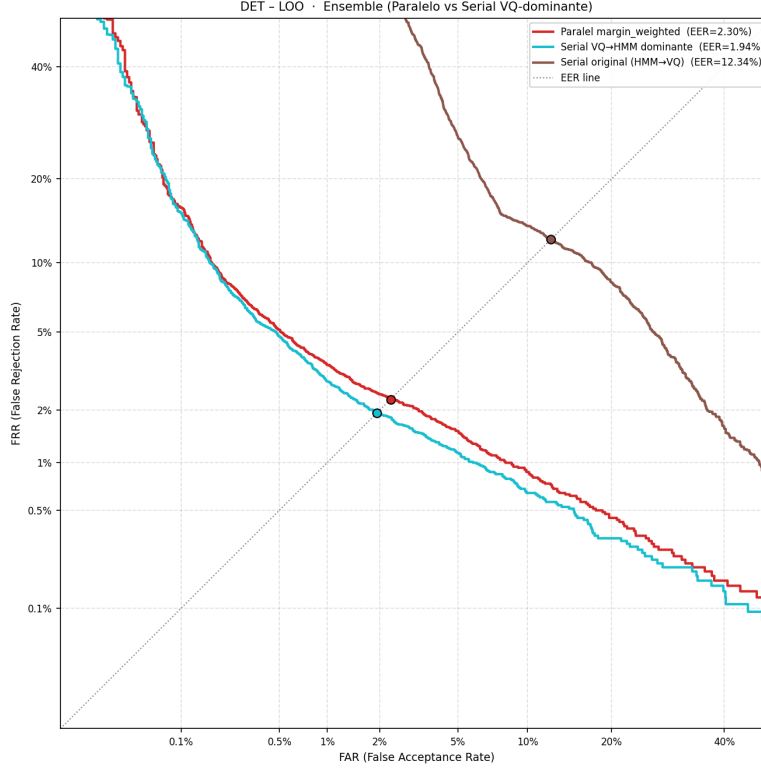


Figura 10: Curvas DET (LOO): sistemas en ensemble.

3.4. Conclusiones de los ensembles

Los resultados confirman que combinar HMM y VQ es una buena estrategia. El ensemble paralelo con `margin_weighted` mejora el EER del mejor modelo individual (VQ opt., 1.90 % en LOO) hasta un 2.30 % en LOO. Este resultado aparentemente peor en LOO se debe a que el GMMHMM del LOO se entrenó con parámetros reducidos (K5-fold, 80 iter., 6 reinicios), lo que limita su contribución al ensemble; en los escenarios con entrenamiento completo (N=74 y N=47) el ensemble sí mejora al VQ opt. de forma clara.

El serial original es un fracaso porque la cadena HMM $\rightarrow$ VQ traslada los errores del HMM al segundo etapa en lugar de corregirlos. La versión VQ-dominante revierte la dirección de la cascada y convierte al VQ en la etapa primaria fiable, usando el HMM solo como respaldo. Con este cambio el serial compite directamente con el paralelo y, en N=47, incluso lo supera ligeramente (1.66 % frente a 1.74 %).